

第 4 卷 数据结构

考试速查：主查数组和区间工具：前缀和/差分、双指针、单调结构、堆、并查集、树状数组、线段树、Sparse Table 和坐标压缩拼接。

Contents

第 4 卷 数据结构	1
考试速查定位	1
DS-00 数据结构路由与接口总表	1
DS-01 前缀和与差分	4
DS-02 树状数组与坐标压缩	6
DS-03 线段树与 Sparse Table	10
DS-04 单调结构与并查集	14
DS-05 线段树进阶	17
DS-06: 双指针与滑动窗口	22
一眼路由	24
模板 1: 最长连续子数组, 和不超过 S	24
模板 2: 最短连续子数组, 和至少 S	24
模板 3: 排序数组两数之和是否存在	25
模板 4: 有序数组原地去重	25
模板 5: 最长无重复字符子串	25
模板 6: 三数之和为 0, 去重计数/列举	26
DS-07 STL 优先的数据结构策略	27

第 4 卷 数据结构

考试速查定位

第 4 卷 数据结构

- **这是第几卷:** 04。
- **主要查什么:** 主查数组和区间工具：前缀和/差分、双指针、单调结构、堆、并查集、树状数组、线段树、Sparse Table 和坐标压缩拼接。
- **什么时候翻:** 区间查询修改、动态排名、前缀和、差分、线段树/树状数组时翻。
- **题面关键词:** 前缀和; 差分; 双指针; 单调栈队列; 堆; DSU; 树状数组; 线段树; Sparse Table。

自动由 03_modules/DS-*.md 重建。定位是把区间、动态维护、窗口、连通性和排名统计整理成可拼接模块。

DS-00 数据结构路由与接口总表

模块编号: DS-00

模块名称: 数据结构路由与接口总表

标签: [数据结构][路由][拼接]

一句话用途: 根据操作类型快速选择 PrefixSum、Difference、树状数组、SegmentTree、SparseTable、MonotonicStack、MonotonicQueue、DSU 或 Compressor。

题面触发词:

- 区间查询。
- 区间修改。
- 单点修改。
- 动态维护。
- 排名、逆序对。
- 合并集合、连通块。
- 滑动窗口最大/最小。
- 连续区间满足和/计数条件。
- 排序数组配对、两数之和、三数之和。
- 最近更大/更小。

什么时候用：

- 题目核心不是复杂推导，而是需要维护某种数据。
- 暴力每次扫描会超时。
- 操作可以归类为查询、修改、合并、统计。

不要什么时候用：

- 数据范围很小，直接暴力更快更稳。
- 题目关键在 DP/图论建模，数据结构只是辅助，此时先完成主模型。
- 动态需求不清楚时，不要一上来写最长的线段树。

复杂度：

- 按模块不同，从 $O(1)$ 、 $O(\log n)$ 到 $O(n \log n)$ 预处理不等。

数据范围参考：

操作规模	暴力风险	优先模块
$n, q \leq 2000$	暴力可能可过	先暴力或前缀和
$n, q \leq 2e5$	每次扫描会 TLE	树状数组 / SegmentTree
值域大但点数少	数组开不下	Compressor + 树状数组/SegmentTree

依赖的标准容器：

- 1-index 数组：上限明确时优先全局静态数组 `a[MAXN]`；上限不清楚时才用 `vector<ll> a(n + 1)`。
- 闭区间：`[1, r]`。
- 坐标压缩后编号也从 1 开始。

输入如何整理：

```
int n, q;
cin >> n >> q;
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口：

```
build(a) // 本卷主模板默认接 1-index vector<ll>
init(n)
add(pos, val)
setv(pos, val)
range_add(l, r, val)
prefix(pos)
query(l, r)
at(pos)
```

考场常数优先口径：

本卷为了拼接统一，很多接口写成 `build(vector<ll>& a)`。

如果题目上限明确、你想直接用全局静态数组，可以把 `build(a)` 改成 `build(n, a)`，内部仍按 `1..n` 循环。

核心不变：数组和查询全部保持 1-index、闭区间 `[1, r]`。

输出能力：

- 静态区间和、动态区间和、区间最值、滑动窗口最值、连通性、排名统计等。

下游可接：

- DP 优化。

- 扫描线。
- 逆序对。
- 动态排名。
- Kruskal。
- 二分答案。

可拼接模块：

题面操作	模块组合
静态区间和	Array + PrefixSum
静态区间最值	Array + SparseTable
单点加 + 区间和	树状数组
区间加 + 最终还原	Difference
区间加 + 单点查	差分树状数组
区间加 + 区间和	双树状数组 或 LazySegmentTree
动态区间 min/max	SegmentTree
值域很大	Compressor + 树状数组/SegmentTree
逆序对	Compressor + 树状数组
连通性/合并	DSU
最小生成树	Graph.edges + DSU + Kruskal
滑动窗口最值	MonotonicQueue
连续区间和/计数满足条件	TwoPointers / SlidingWindow
排序数组配对	TwoPointers
最近更大/更小	MonotonicStack

模板代码：

```
// 数据结构选择口诀：
// 不修改区间和 -> PrefixSum
// 单点改区间和 -> 树状数组 (代码类名 BIT)
// 区间改最终一次输出 -> Difference
// 区间改区间查 -> LazySegmentTree / 双树状数组 (代码类名 RangeBIT)
// 静态最值 -> SparseTable
// 动态最值 -> SegmentTree
// 值域大 -> Compressor first
// 合并集合 -> DSU
```

调用示例：

```
// 静态区间和
PrefixSum ps;
ps.build(a);
cout << ps.query(l, r) << "\n";

// 单点加, 区间和
BIT fw;
fw.build(a);
fw.add(pos, delta);
cout << fw.query(l, r) << "\n";
```

常见坑：

- 树状数组下标不能为 0。
- 坐标范围查询不要直接用 id(L) 和 id(R)，应使用 lower_id/upper_id。
- 前缀和不支持修改。
- SparseTable 不支持修改，且只适合 min/max/gcd 这类可重叠查询。
- 区间 [l, r] 必须统一为闭区间。

暴力/部分替代：

- 静态查询可每次循环求和， $O(nq)$ ，小数据可过。
- 动态修改可直接维护数组并每次扫描，先拿小数据。
- 合并集合可 BFS/DFS 查连通，小数据可过。

升级方向：

- 暴力扫描 -> PrefixSum/树状数组。

- 离散大值域 -> Compressor。
- 树状数组不够表达复杂区间最值 -> SegmentTree。
- 普通 SegmentTree -> LazySegmentTree。

最小测试样例：

n=1
单点区间 l=r
整段区间 l=1,r=n
负数数组
多次修改同一点

DS-01 前缀和与差分

模块编号：DS-01

模块名称：PrefixSum 与 Difference

标签：[数据结构][前缀和][差分][区间]

一句话用途：前缀和用于静态快速查区间和，差分用于批量区间加后一次性还原。

题面触发词：

- 多次询问区间和。
- 数组不变。
- 多次区间加，最后输出结果。
- 对一段 $[l, r]$ 同时增加/减少。

什么时候用：

- PrefixSum：数组不修改，频繁查询区间和。
- Difference：有很多区间加操作，但中间不需要查询完整区间和。

不要什么时候用：

- PrefixSum 不适合有动态修改的区间和。
- Difference 不适合每次修改后马上查区间和。
- 查询 min/max/gcd 不用前缀和。

复杂度：

- PrefixSum：预处理 $O(n)$ ，查询 $O(1)$ 。
- Difference：每次区间加 $O(1)$ ，还原 $O(n)$ 。

数据范围参考：

- $n, q \leq 1e6$ 时仍可用，注意内存和 long long。

依赖的标准容器：

- 1-index vector<ll> a(n + 1)。
- 闭区间 $[l, r]$ 。

输入如何整理：

```
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口：

PrefixSum: build(a), prefix(pos), query(l,r)
Difference: build(a), range_add(l,r,val), restore()

输出能力：

- 区间和。
- 批量区间加后的最终数组。

下游可接：

- 区间 DP 的 cost(l,r)。
- 枚举区间时 $O(1)$ 求和。

- 差分可接扫描线思想。

可拼接模块：

- IntervalDP + PrefixSum。
- BinarySearchAnswer + PrefixSum。
- Difference + final scan。

模板代码：

```
struct PrefixSum {
    int n;
    vector<ll> pre;

    void build(const vector<ll> &a) {
        n = (int)a.size() - 1;
        pre.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
    }

    ll prefix(int pos) {
        if (pos <= 0) return 0;
        if (pos > n) pos = n;
        return pre[pos];
    }

    ll query(int l, int r) {
        if (l > r) return 0;
        return prefix(r) - prefix(l - 1);
    }
};
```

```
struct Difference {
    int n;
    vector<ll> d;

    void init(int n_) {
        n = n_;
        d.assign(n + 2, 0);
    }

    void build(const vector<ll> &a) {
        init((int)a.size() - 1);
        for (int i = 1; i <= n; i++) d[i] = a[i] - a[i - 1];
    }

    void range_add(int l, int r, ll val) {
        l = max(l, 1);
        r = min(r, n);
        if (l > r) return;
        d[l] += val;
        d[r + 1] -= val;
    }

    vector<ll> restore() {
        vector<ll> a(n + 1, 0);
        for (int i = 1; i <= n; i++) a[i] = a[i - 1] + d[i];
        return a;
    }
};
```

调用示例：

```
PrefixSum ps;
ps.build(a);
cout << ps.query(1, r) << "\n";
```

```
Difference df;
df.build(a);
df.range_add(1, r, x);
a = df.restore();
```

常见坑:

- query(1,r) 中 l=1 时要用 pre[0]。
- 区间加时 d[r+1] 需要数组开到 n+2。
- 区间和可能超过 int, 用 ll。
- Difference 中间查询原数组要先还原, 不能直接读 d[i]。

暴力/部分替代:

- 区间和: 每次 for i=1..r 求和。
- 区间加: 每次 for i=1..r 修改数组。

升级方向:

- 前缀和动态修改 -> 树状数组。
- 差分需要在线查询 -> 树状数组差分版或 LazySegmentTree。

最小测试样例:

```
a = [5]
query(1,1) = 5
range_add(1,1,3) 后 a=[8]
```

DS-02 树状数组与坐标压缩

模块编号: DS-02

模块名称: 树状数组、双树状数组与 Compressor

标签: [数据结构][树状数组][坐标压缩]

一句话用途: 树状数组解决单点加与区间和, 坐标压缩把大值域转成可维护的小下标。

命名约定: 正文统一叫“树状数组”; 代码里统一把类名写成 BIT, 短、好抄、不会再出现陌生英文术语。

题面触发词:

- 单点修改, 区间求和。
- 动态前缀和。
- 逆序对。
- 排名统计。
- 值域很大但元素个数不多。
- 区间加, 单点查。
- 区间加, 区间和。

什么时候用:

- 只需要和相关操作, 树状数组比线段树短。
- 坐标值可达 $1e9/1e18$, 但出现次数 $\leq 2e5$ 。

不要什么时候用:

- 树状数组不适合直接维护区间 min/max 的复杂修改。
- 下标可能为 0 且未平移时不能直接用树状数组。
- 坐标范围查询时不要假设端点已经出现在压缩数组里。

复杂度:

- 树状数组: 更新/查询 $O(\log n)$ 。
- Compressor: 排序 $O(n \log n)$, 查询 id $O(\log n)$ 。

数据范围参考:

- $n, q \leq 2e5$ 常用。
- $n \leq 1e6$ 也可用, 注意内存。

依赖的标准容器:

- 1-index 数组。
- 闭区间 $[1, r]$ 。
- 压缩后编号 $1..m$ 。

输入如何整理:

```
vector<ll> all_x;  
// 把所有会出现的坐标 push 到 all_x  
Compressor cp;  
cp.build(all_x);
```

接口:

树状数组: `init(n), build(a), add(pos,val), prefix(pos), query(l,r), at(pos)`
Compressor: `build(xs), id(x), lower_id(x), upper_id(x), val(pos), size()`

输出能力:

- 前缀和。
- 区间和。
- 单点值。
- 压缩编号。

下游可接:

- 逆序对。
- 扫描线。
- DP 优化。
- 动态排名。

可拼接模块:

- Compressor + 树状数组。
- 树状数组 + BinarySearch。
- 差分树状数组 + range add point query。
- 双树状数组 + range add range sum。

模板代码:

```
struct BIT {  
    int n;  
    vector<ll> bit;  
  
    void init(int n_) {  
        n = n_;  
        bit.assign(n + 1, 0);  
    }  
  
    void build(const vector<ll> &a) {  
        init((int)a.size() - 1);  
        for (int i = 1; i <= n; i++) add(i, a[i]);  
    }  
  
    void add(int pos, ll val) {  
        if (pos <= 0 || pos > n) return;  
        for (; pos <= n; pos += pos & -pos) bit[pos] += val;  
    }  
  
    ll prefix(int pos) {  
        if (pos <= 0) return 0;  
        if (pos > n) pos = n;  
        ll res = 0;  
        for (; pos > 0; pos -= pos & -pos) res += bit[pos];  
        return res;  
    }  
  
    ll query(int l, int r) {
```

```

        if (l > r) return 0;
        return prefix(r) - prefix(l - 1);
    }

    ll at(int pos) {
        return query(pos, pos);
    }
};

struct Compressor {
    vector<ll> xs;

    void build(vector<ll> v) {
        xs = v;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
    }

    int id(ll x) {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int lower_id(ll x) {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1;
    }

    int upper_id(ll x) {
        return upper_bound(xs.begin(), xs.end(), x) - xs.begin();
    }

    ll val(int pos) {
        return xs[pos - 1];
    }

    int size() {
        return (int)xs.size();
    }
};

struct BITDiff {
    BIT bit;

    void init(int n) {
        bit.init(n);
    }

    void range_add(int l, int r, ll val) {
        if (l < 1) l = 1;
        if (r > bit.n) r = bit.n;
        if (l > r) return;
        bit.add(l, val);
        bit.add(r + 1, -val);
    }

    ll at(int pos) {
        return bit.prefix(pos);
    }
};

struct RangeBIT {
    int n;
    BIT b1, b2;
};

```

```

void init(int n_) {
    n = n_;
    b1.init(n);
    b2.init(n);
}

void internal_add(int pos, ll val) {
    b1.add(pos, val);
    b2.add(pos, val * (pos - 1));
}

void range_add(int l, int r, ll val) {
    if (l < 1) l = 1;
    if (r > n) r = n;
    if (l > r) return;
    internal_add(l, val);
    internal_add(r + 1, -val);
}

ll prefix(int pos) {
    if (pos <= 0) return 0;
    if (pos > n) pos = n;
    return b1.prefix(pos) * pos - b2.prefix(pos);
}

ll query(int l, int r) {
    if (l > r) return 0;
    return prefix(r) - prefix(l - 1);
}
};

```

调用示例:

```

BIT fw;
fw.build(a);
fw.add(pos, delta);
cout << fw.query(l, r) << "\n";

```

```

Compressor cp;
cp.build(all_x);
BIT bit;
bit.init(cp.size());
bit.add(cp.id(x), 1);
int L = cp.lower_id(left_value);
int R = cp.upper_id(right_value);
cout << bit.query(L, R) << "\n";

```

常见坑:

- 树状数组下标不能为 0。
- 单点 add(pos, val) 中 pos <= 0 或 pos > n 会自动跳过; 区间修改模板会先裁剪到 [1,n], 避免误传 l=0 只改到右边界。
- r + 1 可能是 n + 1, 树状数组的 add 会自动跳过, 但数组大小要正常。
- id(x) 只适合 x 已经在 xs 中; 范围查询用 lower_id/upper_id。
- 双树状数组公式容易错, 确认闭区间 [1,r]。

暴力/部分替代:

- 直接数组修改, 查询时循环求和。
- 逆序对小数据双重循环。

升级方向:

- 树状数组无法处理复杂最值 -> SegmentTree。
- 值域大 -> 先 Compressor。
- 只静态区间和 -> PrefixSum 更简单。

最小测试样例:

```
a = [1,2,3]
query(1,3)=6
add(2,5) 后 query(2,2)=7
坐标 [100, 1000000000] 压缩为 [1,2]
```

DS-03 线段树与 Sparse Table

模块编号: DS-03

模块名称: SegmentTree、LazySegmentTree 与 SparseTable

标签: [数据结构][线段树][懒标记][SparseTable]

一句话用途: 线段树处理动态区间查询/修改, SparseTable 处理静态区间最值或 gcd。

题面触发词:

- 区间最大/最小。
- 单点修改。
- 区间修改。
- 多次查询。
- 静态 RMQ。

什么时候用:

- 树状数组不够处理 min/max 或复杂维护。
- 有动态修改和区间查询。
- 静态区间最值很多次查询时用 SparseTable。

不要什么时候用:

- 只是静态区间和, 用 PrefixSum。
- 只是单点加区间和, 用树状数组更短。
- SparseTable 不支持修改。

复杂度:

- SegmentTree: 建树 $O(n)$, 更新/查询 $O(\log n)$ 。
- LazySegmentTree: 区间修改/查询 $O(\log n)$ 。
- SparseTable: 预处理 $O(n \log n)$, 查询 $O(1)$ 。

数据范围参考:

- $n, q \leq 2e5$ 常见。
- 线段树数组通常开 $4*n + 5$ 。

依赖的标准容器:

- 1-index 数组。
- 闭区间 $[1, r]$ 。

输入如何整理:

```
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口:

SegTreeSum / SegmentTree: build(a), setv(pos,val), add(pos,val), query(l,r)
LazySegTreeSum / LazySegmentTree: build(a), range_add(l,r,val), query(l,r)
SparseTable: build(a), query(l,r)

输出能力:

- 区间和。
- 区间最值。
- 区间加后的区间和。
- 静态 RMQ。

下游可接:

- DP 优化。

- 二分答案。
- 扫描线。

可拼接模块：

- Compressor + SegmentTree。
- DP + SegmentTree。
- Static Array + SparseTable。

模板代码：

```
struct SegTreeSum {
    int n;
    vector<ll> tree;

    void init(int n_) {
        n = n_;
        tree.assign(4 * n + 4, 0);
    }

    void build(int p, int l, int r, const vector<ll> &a) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(p * 2, l, mid, a);
        build(p * 2 + 1, mid + 1, r, a);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    void build(const vector<ll> &a) {
        init((int)a.size() - 1);
        if (n == 0) return;
        build(1, 1, n, a);
    }

    void setv(int p, int l, int r, int pos, ll val) {
        if (l == r) {
            tree[p] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) setv(p * 2, l, mid, pos, val);
        else setv(p * 2 + 1, mid + 1, r, pos, val);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    void setv(int pos, ll val) {
        if (pos < 1 || pos > n) return;
        setv(1, 1, n, pos, val);
    }

    void add(int pos, ll val) {
        ll cur = query(pos, pos);
        setv(pos, cur + val);
    }

    ll query(int p, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        int mid = (l + r) / 2;
        ll res = 0;
        if (ql <= mid) res += query(p * 2, l, mid, ql, qr);
        if (qr > mid) res += query(p * 2 + 1, mid + 1, r, ql, qr);
    }
};
```

```

        return res;
    }

    ll query(int l, int r) {
        l = max(l, 1);
        r = min(r, n);
        if (l > r) return 0;
        return query(1, 1, n, l, r);
    }
};

struct LazySegTreeSum {
    int n;
    vector<ll> tree, lazy;

    void init(int n_) {
        n = n_;
        tree.assign(4 * n + 4, 0);
        lazy.assign(4 * n + 4, 0);
    }

    void build(int p, int l, int r, const vector<ll> &a) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(p * 2, l, mid, a);
        build(p * 2 + 1, mid + 1, r, a);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    void build(const vector<ll> &a) {
        init((int)a.size() - 1);
        if (n == 0) return;
        build(1, 1, n, a);
    }

    void apply(int p, int l, int r, ll val) {
        tree[p] += val * (r - l + 1);
        lazy[p] += val;
    }

    void push(int p, int l, int r) {
        if (lazy[p] == 0 || l == r) return;
        int mid = (l + r) / 2;
        apply(p * 2, l, mid, lazy[p]);
        apply(p * 2 + 1, mid + 1, r, lazy[p]);
        lazy[p] = 0;
    }

    void range_add(int p, int l, int r, int ql, int qr, ll val) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            apply(p, l, r, val);
            return;
        }
        push(p, l, r);
        int mid = (l + r) / 2;
        if (ql <= mid) range_add(p * 2, l, mid, ql, qr, val);
        if (qr > mid) range_add(p * 2 + 1, mid + 1, r, ql, qr, val);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
};

```

```

void range_add(int l, int r, ll val) {
    l = max(l, 1);
    r = min(r, n);
    if (l > r) return;
    range_add(1, 1, n, l, r, val);
}

ll query(int p, int l, int r, int ql, int qr) {
    if (qr < 1 || r < ql) return 0;
    if (ql <= l && r <= qr) return tree[p];
    push(p, l, r);
    int mid = (l + r) / 2;
    ll res = 0;
    if (ql <= mid) res += query(p * 2, l, mid, ql, qr);
    if (qr > mid) res += query(p * 2 + 1, mid + 1, r, ql, qr);
    return res;
}

ll query(int l, int r) {
    l = max(l, 1);
    r = min(r, n);
    if (l > r) return 0;
    return query(1, 1, n, l, r);
}
};

using SegmentTree = SegTreeSum;
using LazySegmentTree = LazySegTreeSum;

// 动态 max/min 只需要把单点线段树里的 merge 和 neutral 换掉。
// ll neutral() { return -LINF; } // max; min 改成 LINF
// ll merge(ll a, ll b) { return max(a, b); } // min 改成 min(a, b)
// 对应替换位置:
// tree[p] = merge(tree[p * 2], tree[p * 2 + 1]);
// query 空贡献 res 从 neutral 开始。

struct SparseTableMin {
    int n, K;
    vector<int> lg;
    vector<vector<ll>> st;

    void build(const vector<ll> &a) {
        n = (int)a.size() - 1;
        if (n == 0) {
            K = 0;
            lg.assign(1, 0);
            st.clear();
            return;
        }
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
        K = lg[n] + 1;
        st.assign(K, vector<ll>(n + 1));
        for (int i = 1; i <= n; i++) st[0][i] = a[i];
        for (int k = 1; k < K; k++) {
            for (int i = 1; i + (1 << k) - 1 <= n; i++) {
                st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
            }
        }
    }

    ll query(int l, int r) {

```

```

        l = max(l, 1);
        r = min(r, n);
        if (l > r) return LINF;
        int k = lg[r - l + 1];
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};

```

调用示例:

```

LazySegTreeSum seg;
seg.build(a);
seg.range_add(l, r, x);
cout << seg.query(l, r) << "\n";

```

```

SparseTableMin st;
st.build(a);
cout << st.query(l, r) << "\n";

```

常见坑:

- 线段树 $4*n+4$ 。
- Lazy 查询前要 push。
- range_add 是闭区间。
- SparseTable 不支持修改。
- SparseTable 查询区间长度 $r-l+1$ 。

暴力/部分替代:

- 单点/区间修改直接改数组。
- 查询直接循环扫描。

升级方向:

- 暴力 -> 树状数组/SegmentTree。
- 静态 min/max -> SparseTable。
- 区间修改 -> LazySegmentTree。

最小测试样例:

```

a=[1,2,3,4]
query(2,3)=5
range_add(2,4,10)
query(1,4)=40
min(2,4)=2 before update

```

DS-04 单调结构与并查集

模块编号: DS-04

模块名称: MonotonicStack、MonotonicQueue 与 DSU

拼接提醒: 本模块和 GRAPH-06 都给了 DSU。考场拼接 Kruskal 时只保留一个 struct DSU, 不要重复复制导致重定义。

标签: [数据结构][单调栈][单调队列][并查集]

一句话用途: 单调栈找最近更大/更小, 单调队列维护滑动窗口最值, 并查集合并集合和查询连通性。

题面触发词:

- 最近更大、最近更小。
- 每个位置左边/右边第一个大于它的数。
- 滑动窗口最大值/最小值。
- 合并集合。
- 判断连通。
- 最小生成树。

什么时候用:

- 单调栈: 每个元素进栈一次, 找附近第一个满足大小关系的位置。

- 单调队列：固定长度窗口或 DP 转移中只需要窗口最大/最小。
- DSU：只需要合并和查询集合，不需要删除。

不要什么时候用：

- 单调栈不适合任意区间查询。
- 单调队列要求窗口按顺序滑动。
- DSU 不支持普通删除边/拆集合。

复杂度：

- 单调栈/队列： $O(n)$ 。
- DSU：近似 $O(1)$ 均摊。

数据范围参考：

- $n \leq 1e6$ 常用，注意数组内存。
- DSU 可用于 $n, m \leq 2e5$ 或更大。

依赖的标准容器：

- 1-index 数组。
- DSU 点编号 $1..n$ 。

输入如何整理：

```
vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];
```

接口：

DSU: `init(n), find(x), unite(a,b), same(a,b)`

输出能力：

- 最近更大/更小位置。
- 每个滑动窗口最大/最小。
- 连通性。
- Kruskal 合并结果。

下游可接：

- 单调栈可接贡献统计。
- 单调队列可接 DP 优化。
- DSU 可接 Kruskal、连通块计数。

可拼接模块：

- Graph.edges + DSU + Kruskal。
- DP + MonotonicQueue。
- Array + MonotonicStack。

模板代码：

```
// 每个 i 左侧最近的严格更大元素位置，不存在为 0
vector<int> previous_greater(const vector<ll> &a) {
    int n = (int)a.size() - 1;
    vector<int> ans(n + 1, 0);
    vector<int> st;
    for (int i = 1; i <= n; i++) {
        while (!st.empty() && a[st.back()] <= a[i]) st.pop_back();
        ans[i] = st.empty() ? 0 : st.back();
        st.push_back(i);
    }
    return ans;
}
```

```
// 滑动窗口最大值，窗口长度 k，返回每个右端点 i 的最大值，i>=k 有意义
vector<ll> sliding_window_max(const vector<ll> &a, int k) {
    int n = (int)a.size() - 1;
    vector<ll> ans(n + 1, 0);
    deque<int> dq;
    for (int i = 1; i <= n; i++) {
```

```

        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k) ans[i] = a[dq.front()];
    }
    return ans;
}

```

```

struct DSU {
    vector<int> fa, sz;

    DSU(int n = 0) {
        if (n > 0) init(n);
    }

    void init(int n) {
        fa.resize(n + 1);
        sz.assign(n + 1, 1);
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        while (x != fa[x]) {
            fa[x] = fa[fa[x]];
            x = fa[x];
        }
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        fa[b] = a;
        sz[a] += sz[b];
        return true;
    }

    bool same(int a, int b) {
        return find(a) == find(b);
    }
};

```

调用示例:

```

auto leftGreater = previous_greater(a);
auto winMax = sliding_window_max(a, k);

```

```

DSU dsu;
dsu.init(n);
dsu.unite(u, v);
cout << (dsu.same(x, y) ? "YES" : "NO") << "\n";

```

常见坑:

- 单调栈中 $\leq$ 和 $<$ 决定严格/非严格。
- 单调队列要先弹出过期下标。
- 滑动窗口答案通常从 $i \geq k$ 才有效。
- DSU 必须 `init(n)`。
- DSU 不能处理删除。

暴力/部分替代:

- 最近更大/更小可向左/右扫描, $O(n^2)$ 。
- 滑动窗口可每个窗口扫一遍。

- 连通性可每次 BFS/DFS。

升级方向：

- 暴力窗口 -> MonotonicQueue。
- 暴力最近元素 -> MonotonicStack。
- BFS 连通查询 -> DSU。

最小测试样例：

```
a=[2,1,3]
previous_greater=[0,1,0]
k=2 sliding max=[-,2,3]
DSU unite(1,2), same(1,2)=true
```

DS-05 线段树进阶

模块编号：DS-05

模块名称：动态开点、可持久化线段树、线段树合并

标签：[数据结构][线段树][动态开点][主席树][可持久化][线段树合并]

一句话用途：在线段树基础上处理超大值域、历史版本查询、多个线段树合并等进阶场景。

题面触发词：

- 值域 $1..1e9$ 或 $1..1e18$ ，但操作次数不多。
- 动态开点线段树。
- 可持久化线段树、主席树、区间第 k 小。
- 每个节点维护一棵权值线段树，需要合并。
- 区间 $chmin/chmax$ 、Segment Tree Beats。

什么时候用：

- 坐标范围巨大，不能开 $4*V$ ，但实际访问点数约 $q \log V$ 。
- 需要保留每个前缀/每次修改后的历史版本。
- 树上/集合中有很多权值线段树，DFS 回来要合并。
- 线段树普通 lazy 无法处理特殊区间最值约束时，再考虑 beats。

不要什么时候用：

- 值域可以离线压缩，普通线段树或树状数组更短。
- 只要静态区间第 k 小且数据很小，排序子数组可拿部分分。
- 没有历史版本需求，不要强行可持久化。
- Beats 难写难调，除非题目明确要求区间取 $\min/\max$ 与和/最值混合维护。

复杂度：

- 动态开点：单次修改/查询 $O(\log V)$ ，空间约 $O(\text{访问次数} * \log V)$ 。
- 可持久化线段树：每次单点更新新建 $O(\log V)$ 个点。
- 区间第 k 小主席树：建 $O(n \log n)$ ，查询 $O(\log n)$ 。
- 线段树合并：总复杂度通常与被创建节点数同阶。

数据范围参考：

- $q \leq 2e5$ ，值域 $\leq 1e9$ ：动态开点常用。
- 主席树节点数估算： $(n + q) * (\log_2(m) + 2)$ 。
- $n \leq 2e5$ ：主席树常开 $4e6$ 左右或用 `vector.reserve`。

依赖的标准容器：

- 1-index 值域闭区间 $[L, R]$ 。
- 节点数组或 `vector<Node>`。
- 坐标压缩数组 `xs`。

输入如何整理：

// 动态开点：保留真实坐标范围

DynamicSegTree seg(1, 1000000000LL);

// 主席树: 先离散化值, 再建前缀版本

```
vector<ll> xs;  
for (int i = 1; i <= n; i++) xs.push_back(a[i]);  
sort(xs.begin(), xs.end());  
xs.erase(unique(xs.begin(), xs.end()), xs.end());
```

接口:

DynamicSegTree: range_add(l,r,val), point_add(pos,val), query(l,r)

PersistentSegTree: update(oldRoot,l,r,pos), kth(rootL,rootR,l,r,k)

MergeSegTree: point_add(root,pos,val), merge(x,y,l,r)

输出能力:

- 超大值域区间加、区间和。
- 静态区间第 k 小。
- 多棵权值线段树合并后的计数/和。

下游可接:

- 扫描线。
- 树上启发式合并。
- 区间第 k 小。
- 权值统计。

可拼接模块:

- DS-05 + DS-02 Compressor。
- DS-05 + TREE/GRAPH DFS。
- DS-05 + DS-03 SegmentTree。

动态开点线段树模板: 区间加、区间和

```
#include <bits/stdc++.h>  
using namespace std;  
using ll = long long;  
  
struct DynamicSegTree {  
    struct Node {  
        int lc = 0, rc = 0;  
        ll sum = 0, lazy = 0;  
    };  
  
    vector<Node> tr;  
    int root;  
    ll L, R;  
  
    DynamicSegTree(ll L_ = 1, ll R_ = 1000000000LL) {  
        init(L_, R_);  
    }  
  
    void init(ll L_, ll R_) {  
        L = L_;  
        R = R_;  
        root = 0;  
        tr.clear();  
        tr.push_back(Node());  
    }  
  
    int new_node() {  
        tr.push_back(Node());  
        return (int)tr.size() - 1;  
    }  
  
    void apply(int p, ll l, ll r, ll val) {  
        tr[p].sum += val * (r - l + 1);  
        tr[p].lazy += val;  
    }  
};
```

```

void push(int p, ll l, ll r) {
    if (tr[p].lazy == 0 || l == r) return;
    ll mid = l + (r - l) / 2;
    if (tr[p].lc == 0) tr[p].lc = new_node();
    if (tr[p].rc == 0) tr[p].rc = new_node();
    apply(tr[p].lc, l, mid, tr[p].lazy);
    apply(tr[p].rc, mid + 1, r, tr[p].lazy);
    tr[p].lazy = 0;
}

void pull(int p) {
    tr[p].sum = 0;
    if (tr[p].lc) tr[p].sum += tr[tr[p].lc].sum;
    if (tr[p].rc) tr[p].sum += tr[tr[p].rc].sum;
}

void range_add(int &p, ll l, ll r, ll ql, ll qr, ll val) {
    if (qr < l || r < ql) return;
    if (p == 0) p = new_node();
    if (ql <= l && r <= qr) {
        apply(p, l, r, val);
        return;
    }
    push(p, l, r);
    ll mid = l + (r - l) / 2;
    range_add(tr[p].lc, l, mid, ql, qr, val);
    range_add(tr[p].rc, mid + 1, r, ql, qr, val);
    pull(p);
}

ll query(int p, ll l, ll r, ll ql, ll qr) {
    if (p == 0 || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return tr[p].sum;
    push(p, l, r);
    ll mid = l + (r - l) / 2;
    return query(tr[p].lc, l, mid, ql, qr) +
           query(tr[p].rc, mid + 1, r, ql, qr);
}

void range_add(ll l, ll r, ll val) {
    if (l > r) return;
    range_add(root, L, R, l, r, val);
}

void point_add(ll pos, ll val) {
    range_add(pos, pos, val);
}

ll query(ll l, ll r) {
    if (l > r) return 0;
    return query(root, L, R, l, r);
}
};

```

可持久化线段树入门：静态区间第 k 小

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

```

```

struct PersistentSegTree {
    struct Node {
        int lc = 0, rc = 0;
    };
};

```

```

    int sum = 0;
};

vector<Node> tr;

PersistentSegTree() {
    tr.push_back(Node());
}

int clone(int p) {
    tr.push_back(tr[p]);
    return (int)tr.size() - 1;
}

int update(int p, int l, int r, int pos) {
    int q = clone(p);
    tr[q].sum++;
    if (l == r) return q;
    int mid = (l + r) / 2;
    if (pos <= mid) tr[q].lc = update(tr[p].lc, l, mid, pos);
    else tr[q].rc = update(tr[p].rc, mid + 1, r, pos);
    return q;
}

int kth(int leftRoot, int rightRoot, int l, int r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2;
    int left_count = tr[tr[rightRoot].lc].sum - tr[tr[leftRoot].lc].sum;
    if (k <= left_count) {
        return kth(tr[leftRoot].lc, tr[rightRoot].lc, l, mid, k);
    }
    return kth(tr[leftRoot].rc, tr[rightRoot].rc, mid + 1, r, k - left_count);
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<ll> a(n + 1), xs;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        xs.push_back(a[i]);
    }
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());

    int m = (int)xs.size();
    PersistentSegTree pst;
    pst.tr.reserve((n + 5) * 20);
    vector<int> root(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        int id = lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin() + 1;
        root[i] = pst.update(root[i - 1], 1, m, id);
    }

    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        if (1 < l || r > n || l > r || k < 1 || k > r - l + 1) {

```

```

        cout << "-1\n";
        continue;
    }
    int id = pst.kth(root[l - 1], root[r], 1, m, k);
    cout << xs[id - 1] << '\n';
}
return 0;
}

```

线段树合并简表:

需求	做法	注意
多棵权值线段树合成一棵	merge(x,y) 返回合并后根	通常会破坏 y, 不能再单独用
子树颜色/权值计数	每个树点一个根, DFS 后合并儿子根	总复杂度看总节点数
只维护出现次数	叶子 sum += sum	最容易写
还要维护最大值位置	合并后 pull 更新答案	tie 规则提前定

线段树合并最小模板: 单点计数

```

struct MergeSegTree {
    struct Node {
        int lc = 0, rc = 0;
        long long sum = 0;
    };

    vector<Node> tr;

    MergeSegTree() {
        tr.push_back(Node());
    }

    int new_node() {
        tr.push_back(Node());
        return (int)tr.size() - 1;
    }

    void point_add(int &p, int l, int r, int pos, long long val) {
        if (p == 0) p = new_node();
        if (l == r) {
            tr[p].sum += val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) point_add(tr[p].lc, l, mid, pos, val);
        else point_add(tr[p].rc, mid + 1, r, pos, val);
        tr[p].sum = tr[tr[p].lc].sum + tr[tr[p].rc].sum;
    }

    int merge(int x, int y, int l, int r) {
        if (x == 0 || y == 0) return x + y;
        if (l == r) {
            tr[x].sum += tr[y].sum;
            return x;
        }
        int mid = (l + r) / 2;
        tr[x].lc = merge(tr[x].lc, tr[y].lc, l, mid);
        tr[x].rc = merge(tr[x].rc, tr[y].rc, mid + 1, r);
        tr[x].sum = tr[tr[x].lc].sum + tr[tr[x].rc].sum;
        return x;
    }
};

```

Segment Tree Beats 低优先级说明:

常见能力：

区间 $chmin$ ：把区间内所有大于 x 的数改成 x 。
区间 $chmax$ ：把区间内所有小于 x 的数改成 x 。
同时查询区间 $sum/max/min$ 。

核心字段示例：

$max1$ 最大值、 $max2$ 严格次大值、 $cntMax$ 最大值出现次数、 sum 区间和。

考场建议：

只有当题面明确出现 $range\ chmin/chmax + sum/max$ 查询，且普通 $lazy$ 做不了时再写。
没背熟不要现场硬造，优先骗部分分。

调用示例：

```
DynamicSegTree seg(1, 1000000000LL);
seg.range_add(10, 20, 3);
cout << seg.query(1, 100) << '\n';

// 主席树区间第 k 小:
int id = pst.kth(root[l - 1], root[r], 1, m, k);
cout << xs[id - 1] << '\n';
```

常见坑：

- 动态开点的 0 是空节点， $tr[0]$ 必须存在且全为 0 。
- mid 用 $l + (r-1)/2$ ，值域 $1e18$ 时避免溢出。
- 动态开点区间长度很大时， $val * (r-l+1)$ 可能爆 $long\ long$ 。
- 主席树查询第 k 小要用 $root[r] - root[l-1]$ 两个版本相减。
- 主席树 k 必须满足 $1 \leq k \leq r-l+1$ 。
- 可持久化更新必须克隆旧点，不能原地改旧版本。
- 线段树合并会破坏被合并的根，后续别再把旧根当独立版本用。
- Beats 不是普通 $lazy$ ，多维护的最大/次大关系错一点就会 WA。

暴力/部分替代：

- 值域大但操作可离线：坐标压缩 + 普通线段树。
- 区间第 k 小小数据：复制子数组排序。
- 历史版本不多：每次复制整个数组或整棵树拿部分分。
- 多集合合并小数据： $map / unordered_map$ 计数。
- Beats 题：分块或暴力扫区间，先拿低档分。

升级方向：

- 动态开点 + 懒标记 $\rightarrow$ 扫描线、区间覆盖。
- 主席树 $\rightarrow$ 带修改主席树，难度较高。
- 线段树合并 $\rightarrow$ 树上权值统计。
- Segment Tree Beats $\rightarrow$ 区间取 min/max 与区间和混合维护。

最小测试样例：

动态开点：

```
add [10,20] += 3
query [1,100] = 33
```

主席树：

```
a = [5,1,4,2,3]
区间 [2,5] 第 3 小 = 3
```

DS-06：双指针与滑动窗口

模块编号：DS-06

模块名称：双指针、相向指针、快慢指针与滑动窗口

标签：双指针、滑动窗口、相向指针、快慢指针、连续区间、排序数组

一句话用途：把双重循环枚举区间/配对降到线性或 $O(n \log n)$ ，常用于连续子数组、排序数组配对、去重和窗口计数。

题面触发词：

- 连续子数组、连续区间、最长/最段子段。
- 正整数数组，区间和满足条件。
- 排序数组，两数之和、三数之和。
- 不重复字符最长子串。
- 去重、原地压缩数组。
- 快慢指针、链表判环。

什么时候用：

- 左右端点都只会单调移动，不会反复回退。
- 数组全为非负数，区间和随右端扩张不减，随左端右移不减。
- 数组已排序或可以先排序，配对关系有单调性。
- 要维护一个连续窗口里的计数、和、种类数。

不要什么时候用：

- 数组有负数时，区间和不再单调，普通滑窗求“和不超过 S”可能错。
- 查询不是连续区间，而是任意子集。
- 每次移动端点后需要复杂区间最值，可能要接单调队列、树状数组或线段树。
- 排序会破坏原下标且题目需要原顺序时，不能直接排序相向指针。

复杂度：

- 同向双指针/滑动窗口：通常 $O(n)$ 。
- 相向双指针：排序后 $O(n \log n)$ ，双指针扫描 $O(n)$ 。
- 三数之和：排序后固定一个数，内层相向指针， $O(n^2)$ 。

依赖的标准容器：

- 普通数组默认 1-index，上限明确时优先静态数组。
- string。
- `array<int, 256>` 或 `vector<int>` 维护字符计数。
- 排序配对常接 `sort`。

输入如何整理：

```
const int MAXN = 200000 + 5;
int n;
cin >> n;
static long long a[MAXN];
for (int i = 1; i <= n; i++) cin >> a[i];
```

字符串窗口：

```
string s;
cin >> s; // 0-index
```

接口：

同向窗口: for r in 1..n expand, while bad shrink l

相向指针: sort(a+1,a+n+1), l=1, r=n, compare sum

快慢指针: slow 记录答案尾部, fast 扫描所有元素

输出能力：

- 最长/最短满足条件的连续区间长度。
- 满足条件的配对数量或一组配对。
- 去重后的长度。
- 字符串最长无重复子串。

下游可接：

- PrefixSum: 有负数时改前缀和 + 哈希/二分。
- MonotonicQueue: 窗口内最大/最小。
- Greedy: 排序后相向配对。
- DP: 把 $O(n^2)$ 枚举前驱优化成窗口。

可拼接模块：

- CPP-003 / CPP-012 排序和二分。
- DS-01 PrefixSum。

- DS-04 MonotonicQueue.
- STR-01 / CPP-011 string.

一眼路由

题面信号	模板	前提
正整数数组，最长和不超过 S	同向滑动窗口	元素非负
正整数数组，最短和至少 S	同向滑动窗口	元素非负
排序数组两数之和	相向指针	有序
三数之和/三元组	固定一个 + 相向指针	先排序
删除有序数组重复项	快慢指针	有序或相邻重复
最长无重复字符串	窗口 + 计数	字符集可计数
固定长度窗口最值	单调队列	查 max/min

模板 1：最长连续子数组，和不超过 S

前提: $a[i] \geq 0$ 。

```
int longest_sum_at_most(const vector<long long> &a, long long S) {
    int n = (int)a.size() - 1;
    int ans = 0;
    int l = 1;
    long long sum = 0;

    for (int r = 1; r <= n; r++) {
        sum += a[r];
        while (l <= r && sum > S) {
            sum -= a[l];
            l++;
        }
        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

调用示例:

```
vector<long long> a = {0, 2, 1, 3, 2};
cout << longest_sum_at_most(a, 4) << '\n'; // 2: 1+3 或 3? 2+1
```

模板 2：最短连续子数组，和至少 S

前提: $a[i] \geq 0$ 。

```
int shortest_sum_at_least(const vector<long long> &a, long long S) {
    int n = (int)a.size() - 1;
    const int INF = 1000000000;
    int ans = INF;
    int l = 1;
    long long sum = 0;

    for (int r = 1; r <= n; r++) {
        sum += a[r];
        while (l <= r && sum >= S) {
            ans = min(ans, r - l + 1);
            sum -= a[l];
            l++;
        }
    }

    return ans == INF ? -1 : ans;
}
```

模板 3：排序数组两数之和是否存在

```
bool two_sum_exists(long long a[], int n, long long target) {
    sort(a + 1, a + n + 1);
    int l = 1;
    int r = n;

    while (l < r) {
        long long sum = a[l] + a[r];
        if (sum == target) return true;
        if (sum < target) l++;
        else r--;
    }
    return false;
}
```

如果要保留原下标：

```
vector<pair<long long, int>> v;
for (int i = 1; i <= n; i++) v.push_back({a[i], i});
sort(v.begin(), v.end());
```

模板 4：有序数组原地去重

输入为 1-index 有序数组 $a[1..n]$ 。

```
int unique_sorted(int a[], int n) {
    if (n == 0) return 0;
    int slow = 1;
    for (int fast = 2; fast <= n; fast++) {
        if (a[fast] != a[slow]) {
            slow++;
            a[slow] = a[fast];
        }
    }
    return slow; // 去重后有效区间为  $a[1..slow]$ 
}
```

STL 版本：

```
sort(a + 1, a + n + 1);
n = unique(a + 1, a + n + 1) - (a + 1); // 新长度
```

模板 5：最长无重复字符串

字符串自然 0-index。

```
int longest_unique_substring(const string &s) {
    vector<int> cnt(256, 0);
    int ans = 0;
    int l = 0;

    for (int r = 0; r < (int)s.size(); r++) {
        unsigned char cr = (unsigned char)s[r];
        cnt[cr]++;

        while (cnt[cr] > 1) {
            unsigned char cl = (unsigned char)s[l];
            cnt[cl]--;
            l++;
        }

        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

模板 6：三数之和为 0，去重计数/列举

```
vector<array<int, 3>> three_sum_zero(int a[], int n) {
    sort(a + 1, a + n + 1);
    vector<array<int, 3>> ans;

    for (int i = 1; i <= n; i++) {
        if (i > 1 && a[i] == a[i - 1]) continue;

        int l = i + 1;
        int r = n;
        while (l < r) {
            long long sum = (long long)a[i] + a[l] + a[r];
            if (sum == 0) {
                ans.push_back({a[i], a[l], a[r]});
                l++;
                r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (sum < 0) {
                l++;
            } else {
                r--;
            }
        }
    }
    return ans;
}
```

常见坑：

- 有负数数组不能直接用“sum 太大就左端右移”的滑动逻辑。
- while 收缩条件写错，会漏掉刚好满足的窗口。
- 相向指针通常要求数组有序；没排序时移动方向没有意义。
- 排序后原下标丢失，需要提前存 {value, id}。
- 字符串窗口用 char 当数组下标时，稳妥写 unsigned char。
- 三数之和去重需要跳过重复的 i/l/r。

暴力/部分分替代：

- 区间问题小数据：双重循环枚举 [l, r]，每次累计和。
- 两数之和小数据：双重循环枚举 i, j。
- 字符串小数据：枚举每个左端，向右用 set 检查重复。

升级方向：

- 双重循环区间和 -> 正数数组滑动窗口。
- 双重循环配对 -> 排序 + 相向指针。
- 固定窗口最值 -> 单调队列。
- 有负数的区间和 -> PrefixSum + 哈希/二分/数据结构。

最小测试样例：

最长和不超过 S：

a = [2, 1, 3, 2], S=4 -> 2

最短和至少 S：

a = [2, 1, 3, 2], S=5 -> 2

两数之和：

[1, 2, 4, 7], target=6 -> true

最长无重复子串：

abca -> 3

DS-07 STL 优先的数据结构策略

模块编号: DS-07

模块名称: STL 优先的数据结构策略

标签: [数据结构][STL][priority_queue][unordered_map][set][静态数组][考试速写]

一句话用途: 明确哪些数据结构考场上直接用 STL, 哪些才需要手写模板, 并整理速度写法、比较器、哈希表和 noexcept 的常见坑。

题面触发词:

- 堆、优先队列、每次取最小/最大。
- 队列、栈、双端队列。
- 集合、映射、计数、前驱后继。
- 哈希表、状态缓存、字符串计数。
- 需要快速编码, 不想手写复杂容器。

什么时候用:

- 需要的容器行为 STL 已经直接支持。
- 算法核心不在容器实现, 而在建模、转移、遍历。
- 想减少手写代码量和 bug 面。

不要什么时候用:

- 需要区间和/区间最值/区间修改, STL 容器不直接支持, 转 树状数组/SegmentTree。
- 需要第 k 小/动态排名, 普通 set 不够, 转坐标压缩 + 树状数组。
- 需要堆中任意删除或修改 key, 普通 priority_queue 不支持, 常用“懒删除”或改用 set。
- 哈希表被卡或需要稳定最坏复杂度时, 优先 map/set。

复杂度:

- 全局静态数组随机访问 $O(1)$, 常数小, 适合题目给定上限的数组/DP 表。
- vector 随机访问 $O(1)$, 尾插均摊 $O(1)$, 适合规模运行时才知道或需要动态增长的序列。
- queue/stack/deque 常用端点操作 $O(1)$ 。
- priority_queue push/pop $O(\log n)$, top $O(1)$ 。
- set/map/multiset 插入删除查找 $O(\log n)$ 。
- unordered_map/unordered_set 平均 $O(1)$, 最坏可能退化。

数据范围参考:

- 上限明确: 数组、DP 表、距离表优先全局静态数组。
- $n \leq 2e5$: priority_queue/set/map/unordered_map 这类 STL 行为容器通常稳。
- $n \geq 1e6$: 少用 map; 哈希表要 reserve; 线性表优先静态数组。
- 状态数明确且范围小: 优先数组 memo, 不要用 map。

依赖的标准容器:

- 全局静态数组。
- vector、array、string。
- queue、stack、deque、priority_queue。
- set、multiset、map、unordered_map、unordered_set。

输入如何整理:

```
const int MAXN = 200000 + 5;
ll a[MAXN];           // 1-index 数组, 题目给上限时优先这样写
queue<int> q;          // BFS
deque<int> dq;         // 单调队列 / 0-1 BFS
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> > pq; // Dijkstra
unordered_map<long long, int> mp; // 编码状态计数
```

接口:

```
static array: a[1..n], dp[1..n][...]
vector: reserve, resize, assign, push_back, pop_back, back
priority_queue: push, top, pop, empty
set/map: insert, erase, find, count, lower_bound, upper_bound
unordered_map: reserve, max_load_factor, find, count, operator[]
```

输出能力:

- 堆顶最大/最小。

- 当前有序集合前驱后继。
- key 到计数/答案的映射。
- BFS/DFS/DP 状态容器。

下游可接：

- Dijkstra、Kruskal、Topo、BFS。
- 记忆化搜索。
- 贪心、扫描线、离线处理。
- 坐标压缩 + 树状数组。

可拼接模块：

需求	优先 STL	什么时候换手写/专门结构
数组、DP 表、距离矩阵	全局静态数组	上限不清楚或必须动态增长才用 vector
邻接表	优先标准 Graph；短题可写 vector<pair<int,ll>> g[MAXN]	不需要 edges/input_id 时才用临时静态邻接表
BFS	queue	双端权值 0/1 转 deque
0-1 BFS / 单调队列	deque	无
每次取最小/最大	priority_queue	要任意删除改 set 或懒删除
前驱后继、有序去重	set/multiset	要排名转 树状数组
key-value 映射	map/unordered_map	key 范围小转 vector
字符串	string	多模式匹配转 Trie/AC

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Node {
    ll dist;
    int id;
};

struct NodeCmp {
    bool operator()(const Node &a, const Node &b) const noexcept {
        if (a.dist != b.dist) return a.dist > b.dist; // 小 dist 优先
        return a.id > b.id;
    }
};

struct PairHash {
    size_t operator()(const pair<int, int> &p) const noexcept {
        return ((uint64_t)(unsigned)p.first << 32) ^ (unsigned)p.second;
    }
};

const int MAXN = 200000 + 5;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    static int a[MAXN];
    for (int i = 1; i <= n; i++) cin >> a[i];

    priority_queue<Node, vector<Node>, NodeCmp> pq;
    pq.push({0, 1});
```

```

unordered_map<pair<int, int>, int, PairHash> memo;
memo.max_load_factor(0.7);
memo.reserve(n * 2 + 1);

return 0;
}

```

调用示例:

```

// 堆不能删除旧状态: 用懒删除
while (!pq.empty()) {
    auto cur = pq.top();
    pq.pop();
    if (cur.dist != dist[cur.id]) continue;
}

// unordered_map 判断存在, 不要用 mp[key] 误创建
auto it = memo.find({x, y});
if (it != memo.end()) {
    cout << it->second << '\n';
}

```

常见坑:

- reserve(n) 只预留容量, 不改变 size(), 不能直接 a[i]。
- 访问 front/back/top 前必须 empty() 检查。
- priority_queue 默认最大堆; 小根堆用 greater<pair<...>> 或自定义比较器。
- priority_queue 的比较器含义是“谁优先级更低返回 true”, 写反很常见。
- sort 比较函数必须是严格弱序, 不能写 <=。
- unordered_map 没有顺序, 也没有 lower_bound。
- mp[key] 会创建默认值; 只查存在用 find/count。
- multiset.erase(x) 会删掉所有 x; 只删一个要先 find, 存在再 erase(it)。
- noexcept 不是必须, 但简单的自定义 hash / comparator 可以加; 不要为了加 noexcept 写复杂代码。真正重要的是比较器不抛异常、不修改外部状态、逻辑稳定。

暴力/部分替代:

- 小数据每次线性扫描找最小/最大, 先替代堆。
- 小数据用 vector<pair<K,V>> 扫描查 key, 先替代 map。
- 需要排名但不会树状数组时, 先 vector 排序重算拿部分分。

升级方向:

- vector 扫描 -> priority_queue/set/map。
- map 状态缓存 -> 数组 memo 或编码后 unordered_map。
- set 排名需求 -> Compressor + 树状数组。
- priority_queue 任意删除需求 -> 懒删除 / multiset。

最小测试样例:

堆: push 3,1,2, 小根堆 top=1

哈希: memo[{1,2}]=5 后 find({1,2}) 存在

multiset: 插入 5,5, find 后 erase 一个迭代器, 还剩一个 5